

From Best Practices to Evidence How Software Heritage and SWHID Support Trustworthy Open Source

Wendi Urribarri

Process & Quality Engineer at Woven by Toyota
Software Heritage Ambassador (Industry vertical)
Contributor to the ELISA Lighthouse OSS SIG

My goal today: Connect the dots



1



I'm a **Software Heritage Ambassador**
since February 2026 [link](#)

2

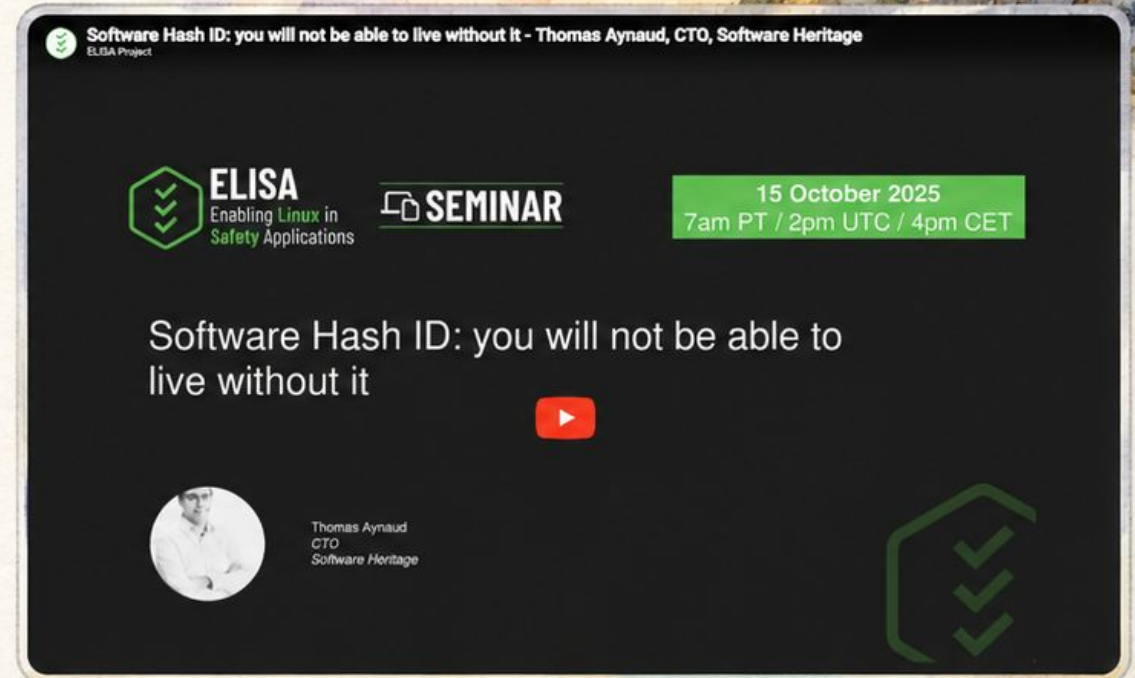


I'm a contributor to the
Lighthouse OSS SIG [link](#)
since June 2025

3



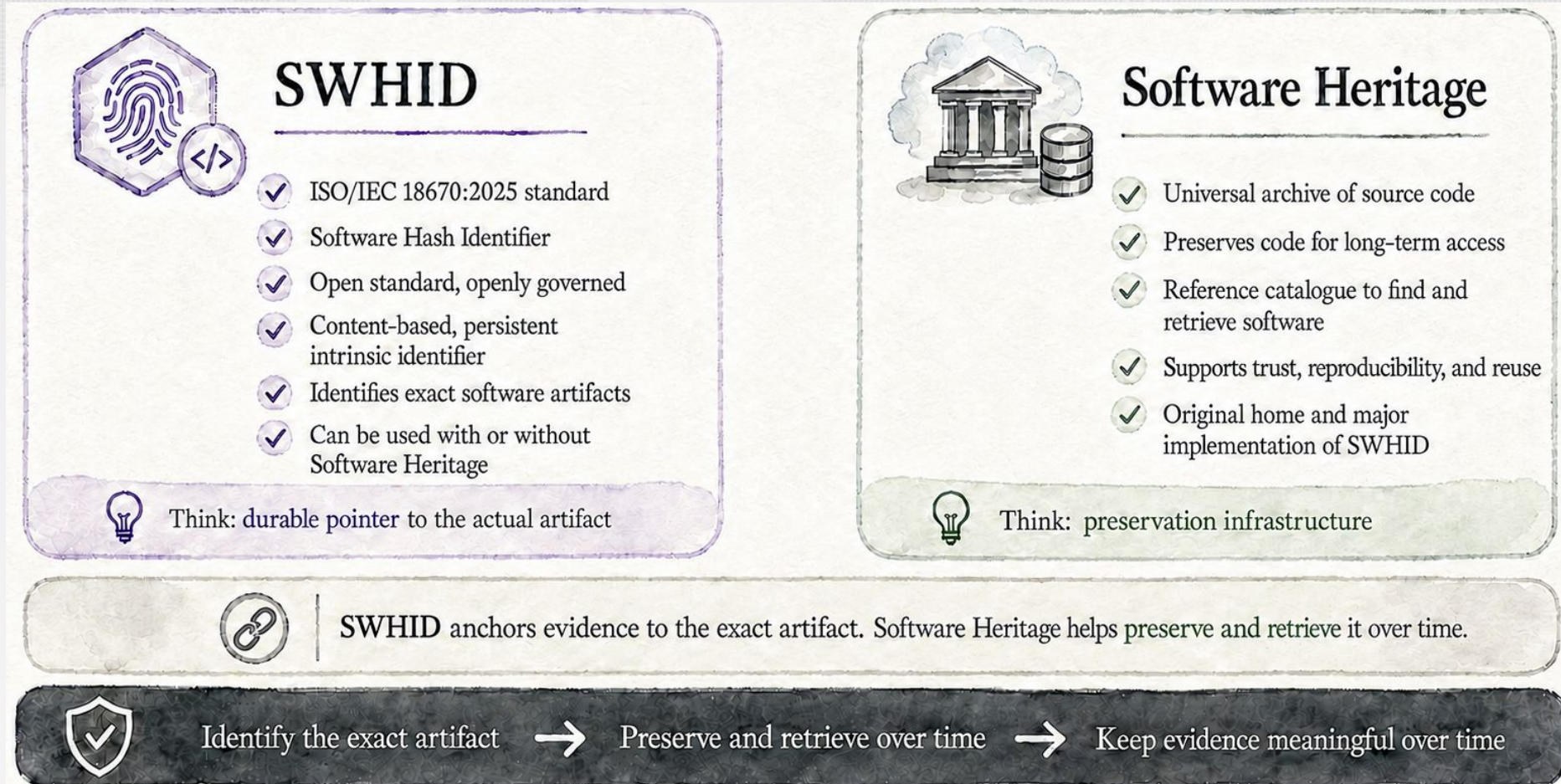
Thomas Aynaud's ELISA webinar [link](#)
already provided a broad introduction
about Software Heritage and SWHID



Now, let's explore how **Software Heritage** and **SWHID** can support the kind of OSS quality and best-practice assessment discussions we are having in ELISA.

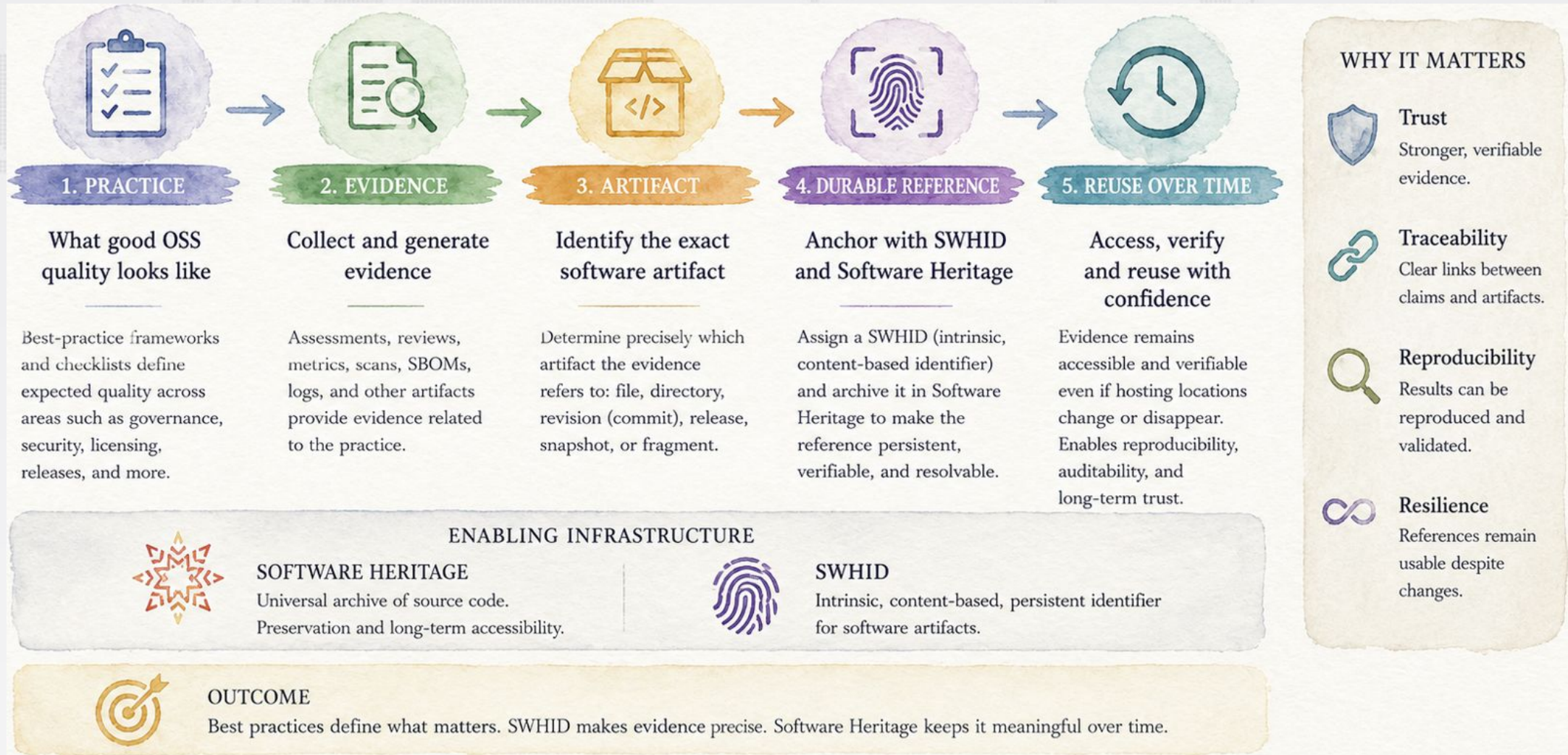


SWHID and Software Heritage: related, but not the same

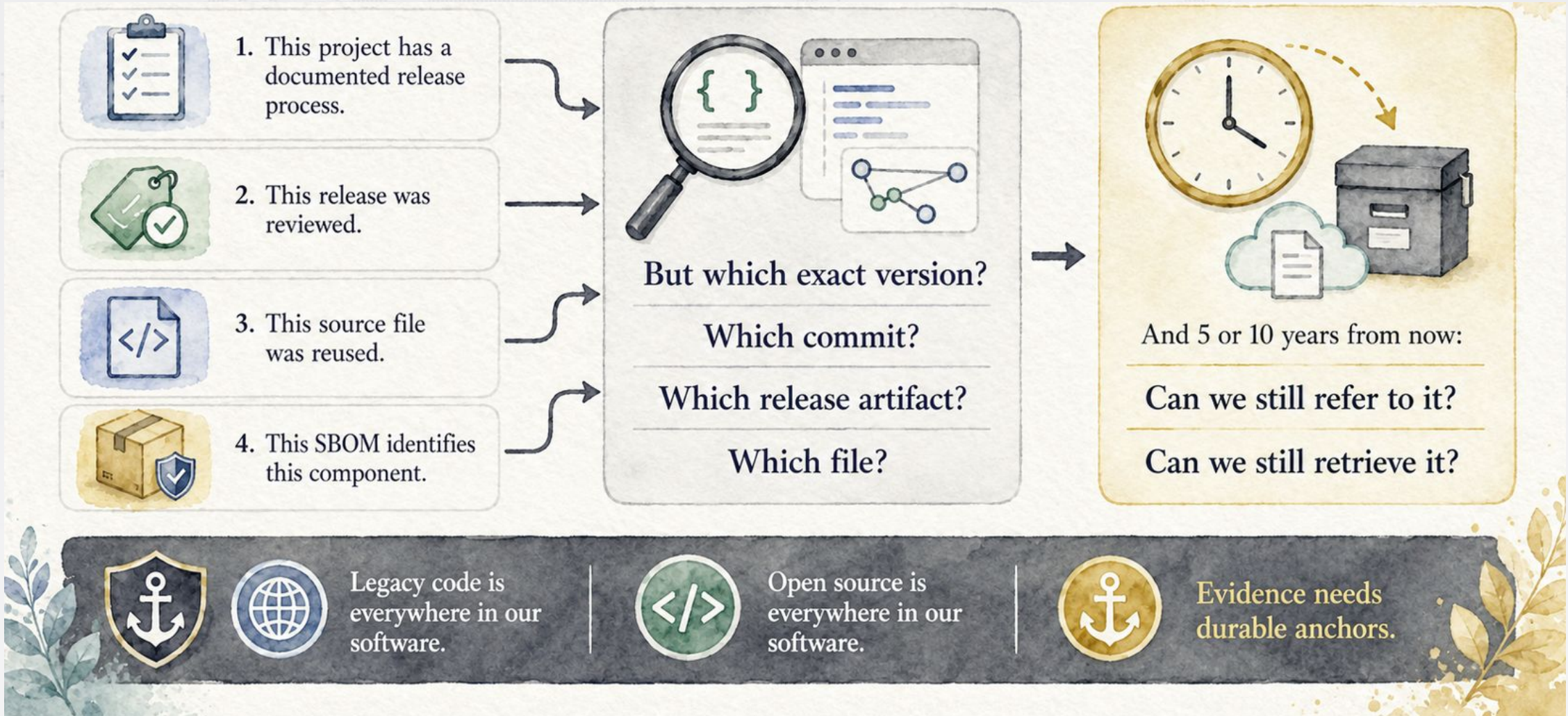




The evidence *lifecycle*



The evidence *question*

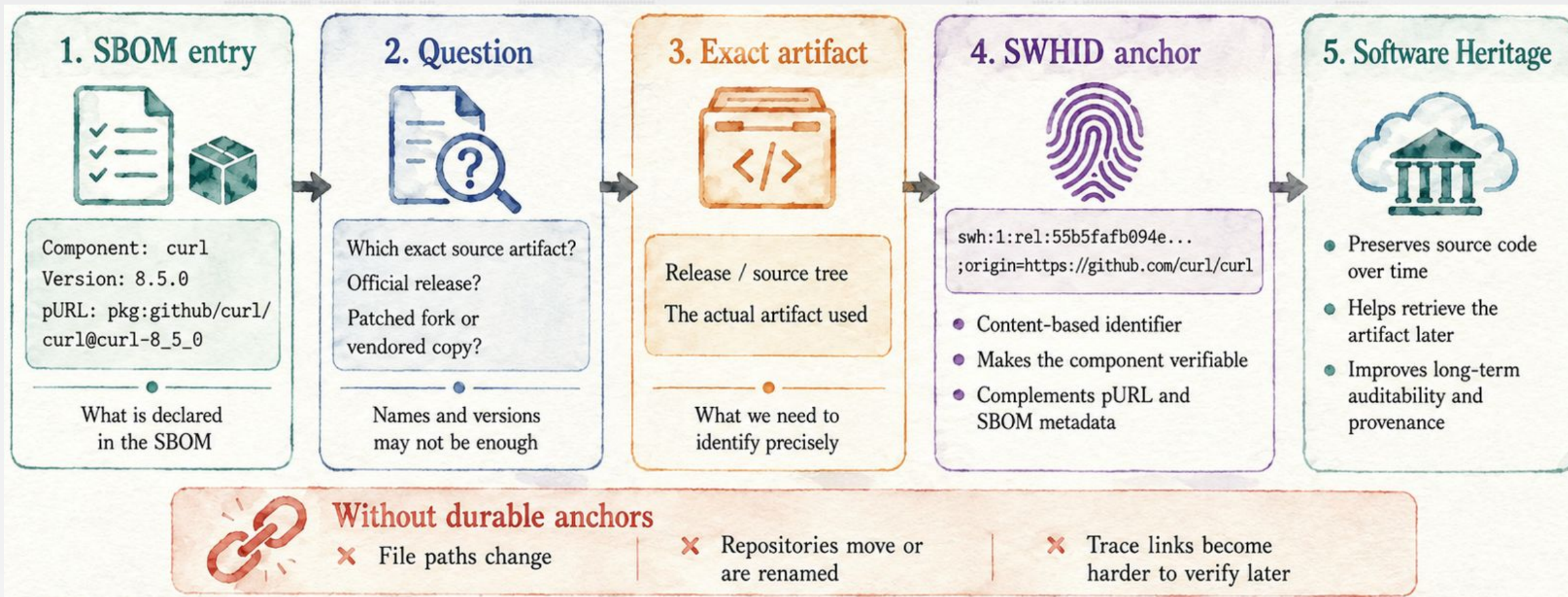


From best practices to durable evidence

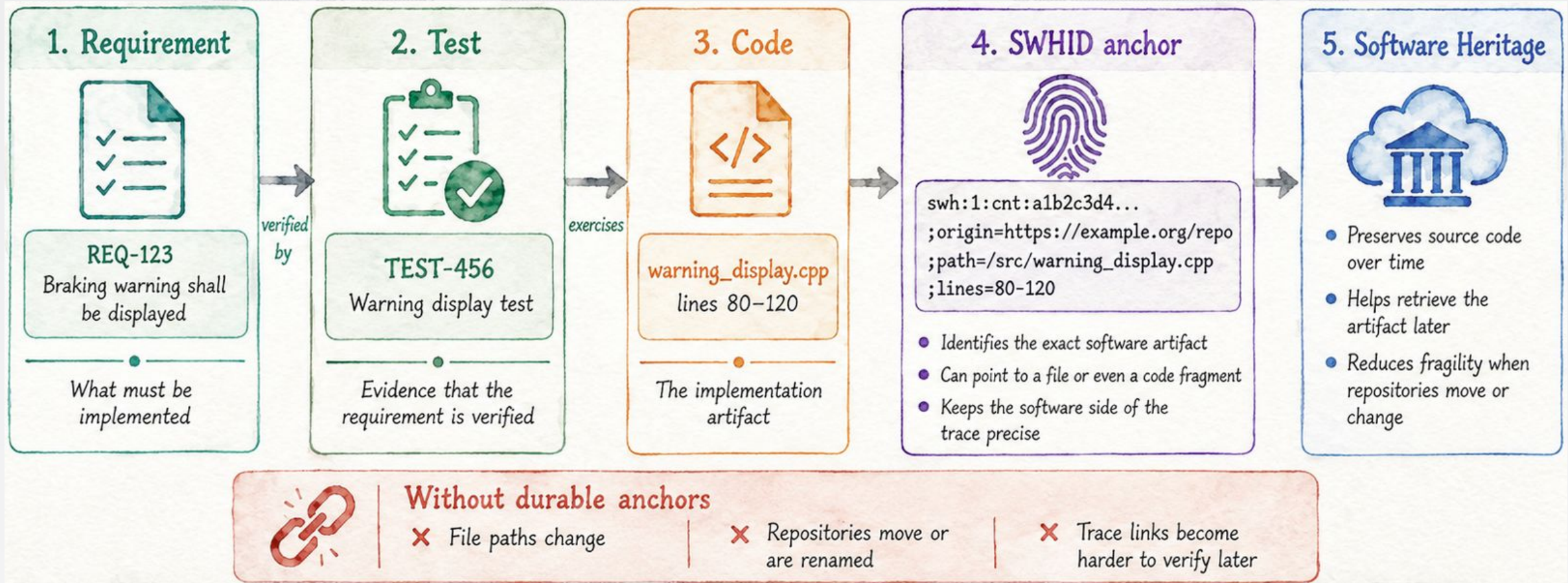


PRACTICE AREA	ASSESSMENT EVIDENCE	SWHID CONTRIBUTION
 Release management	Release X was assessed	Identify the exact release artifact
 Code review	Review evidence for change Y	Identify the exact revision or file
 Licensing	License scan result	Identify the exact source artifact scanned
 SBOM	Component listed in SBOM	Make the component verifiable
 Reuse	Downstream uses upstream artifact	Identify what was actually reused
 Traceability	Requirement/test/code relation	Anchor trace links to stable artifacts
 Best-practice evidence becomes stronger when it can be anchored to exact software artifacts.		

Example: SBOMs



Example: Traceability reqs/tests/code





Same pattern, different frameworks

Illustrative ways Software Heritage and SWHID can strengthen evidence across different OSS best-practice programs

Program	Practice	Claim	Evidence	Artifact	SWH/SWHID contribution	Long-term value
  ELISA Lighthouse OSS Best Practices	Limit and monitor dependencies	Project identifies and monitors external dependencies	Dependency inventory or SBOM, policy, review/update records	Exact upstream components and versions used	Anchor exact dependency artifacts with SWHIDs; Software Heritage helps retrieve them later	Supports vulnerability follow-up, reuse, and verification of what was actually used
  OpenSSF Best Practices Badge	license_location	Project posts license(s) in a standard repository location	LICENSE file(s), repository path, badge review evidence	Exact license file(s) and assessed revision or release	Anchor the exact license file and source tree or revision; preserve access if the repository moves	Makes license evidence durable for audits, reuse, and compliance
  Eclipse Trustable Software Framework	TA-MISBEHAVIOURS	Prohibited misbehaviours are identified, and mitigations are specified, verified, and validated	Analysis, mitigation specification, verification and validation records, linked code/tests	Exact analyzed release or revision, mitigation code, and test artifacts	Anchor code and test artifacts and the analyzed source snapshot; keep evidence retrievable	Strengthens trust arguments and preserves traceable evidence for later reassessment
  Apache Project Maturity Model	RE50	Project documents a repeatable release process so a newcomer can independently generate release artifacts	Release process documentation, scripts, build instructions, source tag, release artifacts	Exact release process docs, source tag or revision, and release artifact set	Identify the exact release tag, source tree, and artifacts; preserve the source used to reproduce the release	Helps future contributors reproduce releases and understand what was released

Leaving with a clear **signal**

Best practices set the direction.

Evidence shows the work.

SWHID anchors the truth.

Software Heritage keeps it alive.



Make software
evidence precise



Keep it accessible
over time



Build trust
at scale



Software Heritage

The universal archive
of source code



1. Identify what matters

Frameworks and programs define practices and the evidence that matters.

What should we prove?



2. Connect to the exact artifact

SWHID links evidence to the precise software artifact: release, revision, file, directory, or fragment.

What exactly are we talking about?



3. Preserve and access

Software Heritage preserves source code so that artifacts remain accessible and understandable.

Can we still access it tomorrow?



4. Build trust, enable reuse

Durable evidence enables audits, compliance, security, and reuse across projects and generations.

Can others trust and build on it?

Thank you!



Software Heritage

THE GREAT LIBRARY OF SOURCE CODE

Thank you for your attention!

